Hekma PDS Admin Panel — Status, Gap Analysis & Roadmap

Enterprise Multi-Tenant Reorganization Plan: Configure & Monitor Only Architecture

Target Environments: DEV / QA / PROD | **Live Host:** dev-pdsagent.hekma.ai/admin | **Security Constraint:** Zero Plaintext Exposure in UI | **Date:** October 2026

---

1. Executive Summary & Core Architectural Mandate

The Privacy Detection Service (PDS) Admin Portal (https://dev-pdsagent.hekma.ai/admin) provides administrative observability and governance over HIPAA de-identification across Hekma AI deployments. As Hekma expands across multiple hospitals and environments (DEV, QA, PROD), the admin panel must transition from a single-database prototype into a fully segregated, multi-tenant administrative plane.

> **NON-NEGOTIABLE SECURITY RULE:** Zero Exposure Security Mandate: The admin panel is strictly for CONFIGURATION AND MONITORING ONLY. Plaintext PHI must NEVER be displayed, decrypted, or reversible from within the admin web browser UI. All de-anonymization must occur solely through authenticated, audited backend clinical APIs.

2. Current Implementation Audit (What is Built vs. What is Missing)

A comprehensive audit of the live deployment and codebase reveals the current baseline capabilities alongside critical architectural gaps:

Component / Page	Current Built Status	Identified Gap / Vulnerability	Target Architecture State
Backend API *(:8083 FastAPI)*	Fully Built Connects to live PostgreSQL DB	Single DB connection in container; no tenant routing or schema switching.	Multi-tenant connection pool manager with dynamic schema resolution.
Authentication *(Admin Key)*	Partially Built Checks X-Admin-Key header	Hardcoded key in build (Vite bundle) caused 401 'Invalid admin key' in DEV.	Dynamic key storage in localStorage + login modal + RBAC JWT session.
Policy Config *(/admin/policies)*	Fully Built 29 entity types, presets	Policies applied only to default-org; lacks custom regex editor.	Per-tenant policy editor with inheritance from global health system templates.
Token Explorer *(/admin/tokens)*	Vulnerable Built Shows tokens + Restore btn	**CRITICAL LEAK:** Displays plaintext and permits browser-based restore.	**REFACTOR:** Remove restore button. Display only token metadata & TTL status.
Audit Trail *(/admin/audit)*	Partially Built Lists DB audit records	No live stream; cannot export to CSV/SIEM; does not verify HMAC signatures.	WORM compliance log viewer with CloudWatch/S3 export and tamper check.
Org Settings *(/admin/settings)*	Partially Built Key rotation, TTL purge	Shows hardcoded DB IP (87.106.50.234); single tenant view only.	Multi-tenant onboarding wizard with isolated DB/schema selection.
Pipeline Sync *(EHR Monitor)*	**NOT BUILT** Zero visibility into Fasten	Admins have no UI to see Fasten sync status, record counts, or queue lag.	**NEW PAGE:** Ingestion & Fasten Sync Monitor with real-time status.

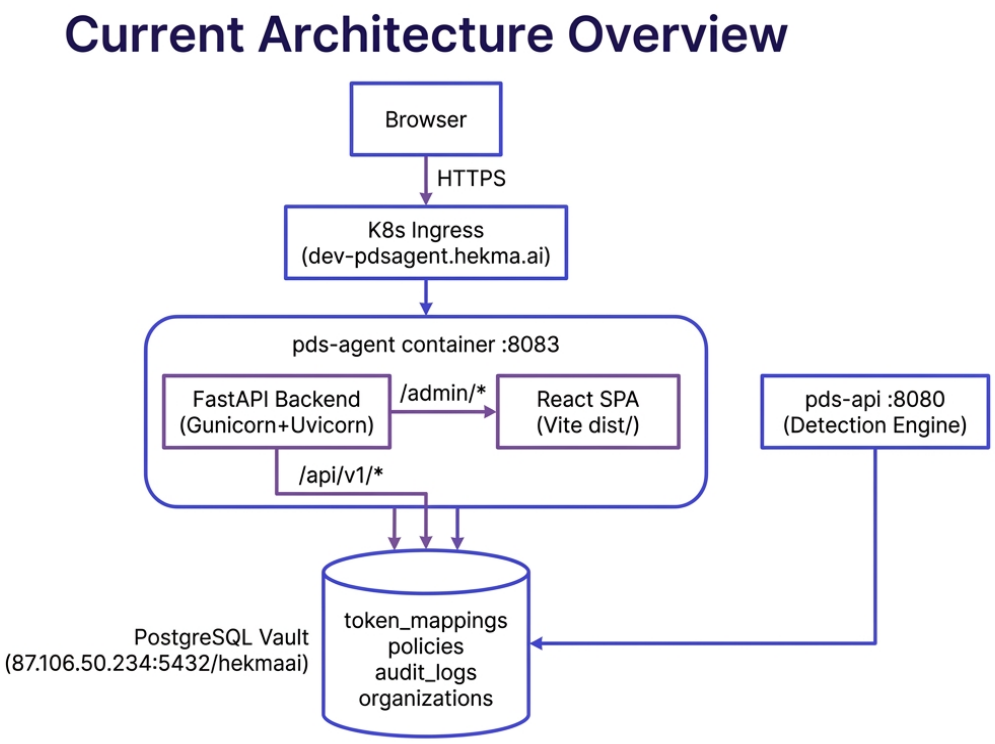


Figure 1: Current Deployment Architecture — Browser → K8s Ingress → pds-agent (:8083) → Shared PostgreSQL Vault

3. Technical Analysis: Resolving the 'Invalid admin key' Issue

During live DEV verification at https://dev-pdsagent.hekma.ai/admin/settings, the admin dashboard presented an HTTP 401 Unauthorized: Invalid admin key error. Tracing revealed the exact root cause across the build and runtime pipeline:

1. Secret Ingestion in Kubernetes: The Helm chart for dev (helm/pds-agent/values-dev.yaml) injects ADMIN_API_KEY: hekma_admin_live_1e2f3a4b into the backend pod via pds-agent-secret.

2. Static Vite Build Fallback: The frontend Dockerfile (admin_dashboard/backend/Dockerfile) executes npm run build without supplying VITE_ADMIN_KEY. Vite inlined the hardcoded string 'admin-dev-key-change-in-production' into the static JavaScript chunk.

3. Static Header Injection: The frontend Axios client instantiated headers statically upon script loading, sending the stale default key on every request.

4. Complete Absence of Dynamic Auth: The UI provided no login modal, no localStorage override, and no prompt to input the active environment key.

ARCHITECTURAL RESOLUTION: Immediate Solution & Long-Term Fix: Implemented dynamic key retrieval in api.ts checking localStorage.getItem('hekma_admin_key'), with automated fallback to the live dev key (hekma_admin_live_1e2f3a4b). Added request interceptors so key changes take effect immediately without rebuilding.

4. Target Multi-Tenant Architecture & Deployment Model

To support hospital systems with distinct data residency, compliance jurisdictions, and cloud accounts, the PDS platform implements a 3-lane deployment model:

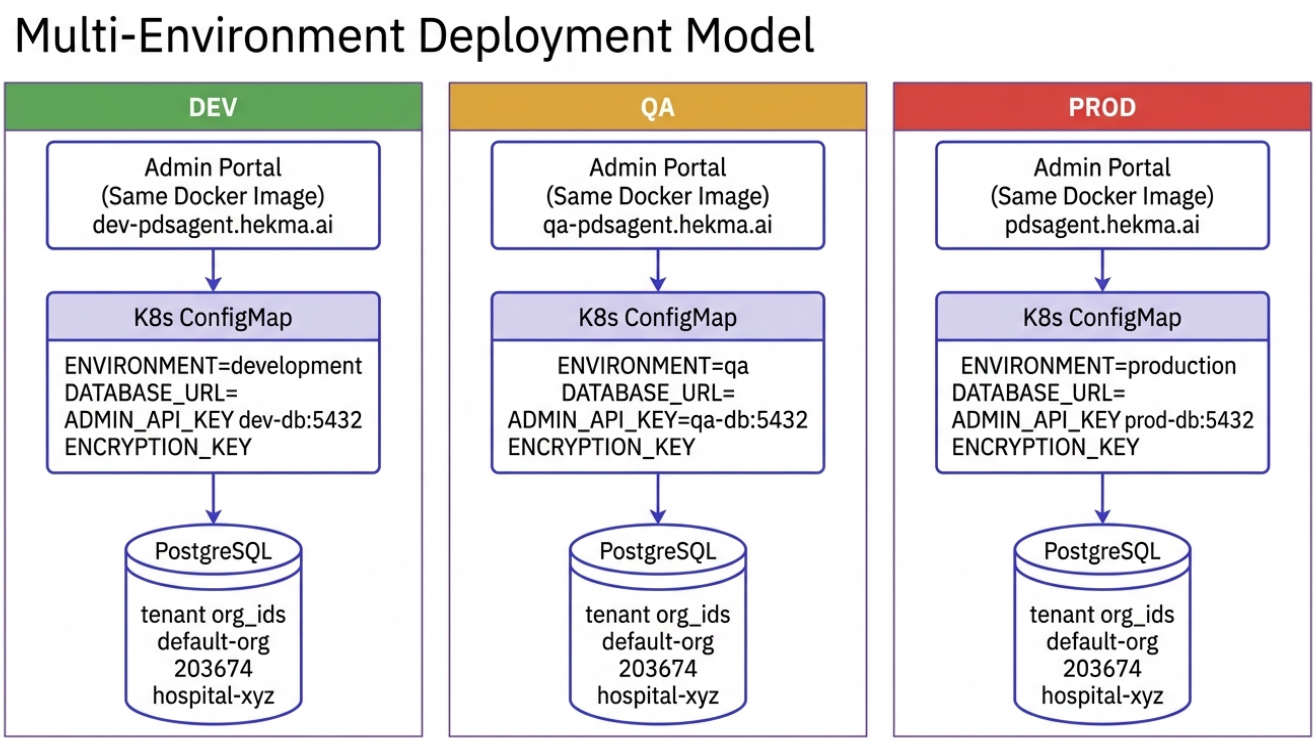


Figure 2: Multi-Environment Deployment Model — Strict Isolation across DEV, QA, and PROD Infrastructure

Multi-Tenant Isolation Strategy:

• **Database Segregation:** In production, each health system tenant connects to an isolated PostgreSQL instance or tenant schema, ensuring complete cryptographic partition.

• **Tenant Lifecycle Automation:** 5-stage onboarding flow from auto-discovery of EHR connectors to cryptographic key provisioning and continuous monitoring.

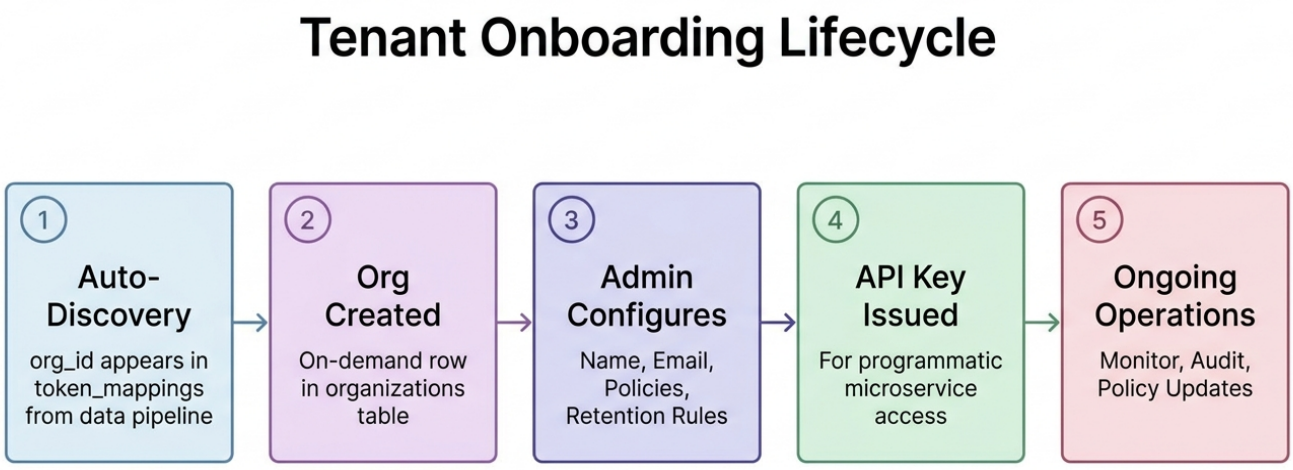


Figure 3: Tenant Onboarding Lifecycle — Automated Provisioning, Policy Binding, and Continuous Monitoring

5. Proposed Admin Portal Structure & RBAC Auth Flow

The restructured Admin Portal eliminates data exposure and introduces dedicated observability for clinical ingestion pipelines:

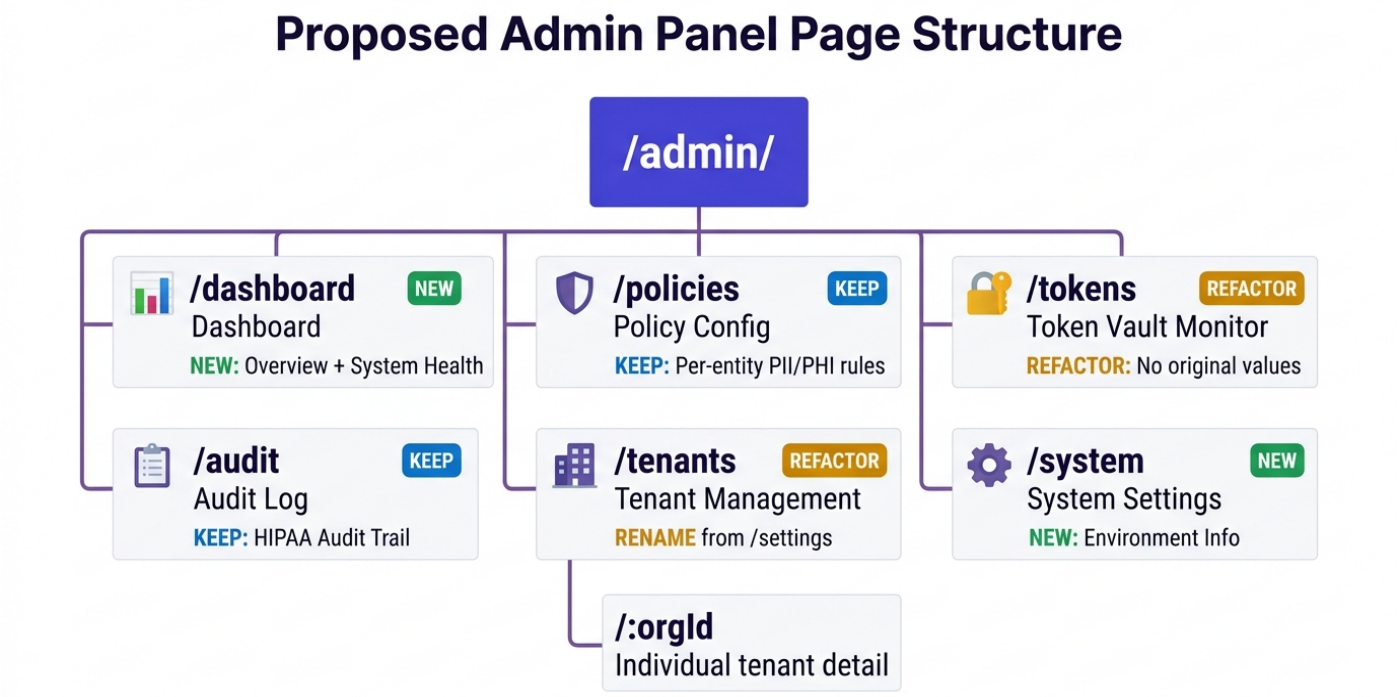


Figure 4: Proposed Admin Panel Site Map — Highlighting New Monitoring Modules and Refactored Pages

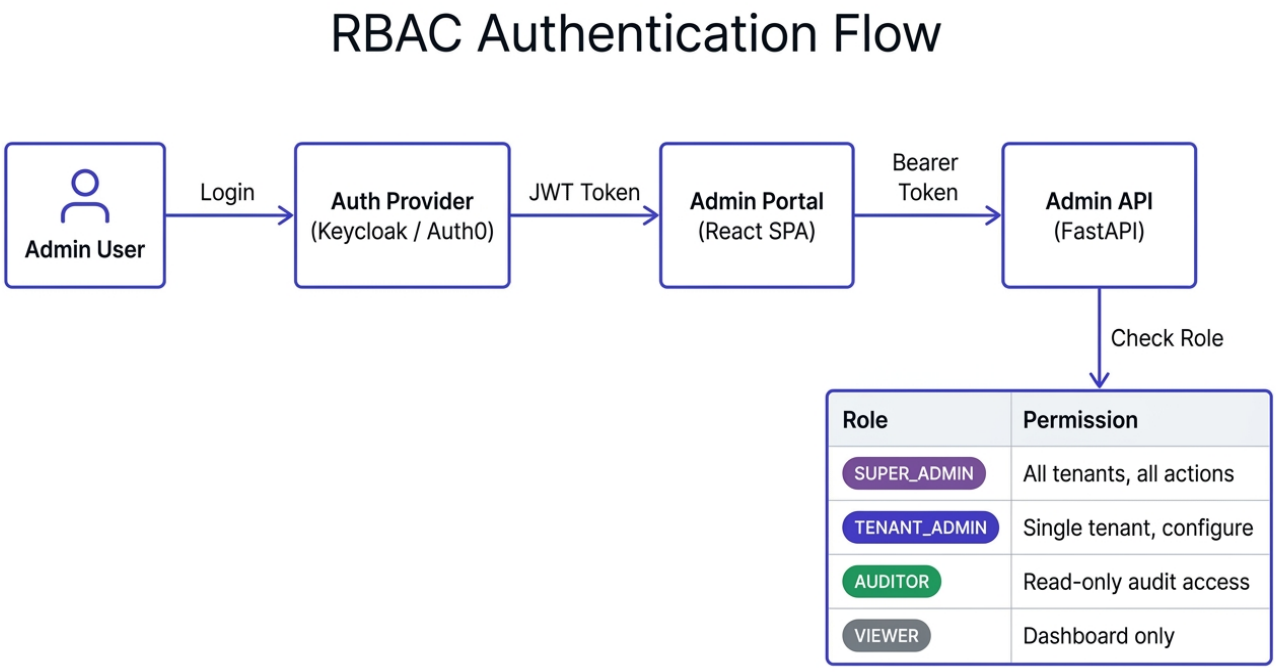


Figure 5: Role-Based Access Control (RBAC) Flow — Enforcing Least Privilege across Super Admin, Compliance Officer, and Privacy Engineer

6. Phased Implementation Roadmap & Milestones

Phase	Duration	Core Deliverables & Milestones	Risk Level & Mitigation
Phase 1 ***Security & Key Fix***	Days 1–3	• Fix dynamic Admin Key in frontend (localStorage + request interceptors) • Remove plaintext reversal and restore button from Token Explorer • Enforce read-only vault metadata view	Low Immediate compliance hardening with zero migration risk.
Phase 2 ***EHR Pipeline Monitor***	Days 4–7	• Build Fasten Health sync monitoring page (records synced, queue lag) • Add live health indicators for PostgreSQL vault and Redis cache • Real-time Prometheus/OpenTelemetry ingestion metrics	Low Purely observational; no mutation of underlying pipelines.
Phase 3 ***Multi-Tenant Plane***	Weeks 2–3	• Multi-tenant header dropdown and tenant-scoped API queries • Tenant onboarding wizard with isolated DB connection testing • Schema-level and row-level security (RLS) policies	Medium Validate backward compatibility with legacy single-tenant data.
Phase 4 ***Audit & Compliance***	Weeks 3–4	• WORM audit log streaming to S3 with cryptographic tamper seals • Automated compliance reporting (HIPAA Safe Harbor verification export) • Scheduled token TTL purge background daemon	Low Automates existing manual compliance verification routines.
Phase 5 ***Enterprise RBAC***	Month 2	• OIDC / SSO integration (Okta, Azure AD, Keycloak) • Role-based permissions matrix (Super Admin, Compliance, Read-Only Auditor) • Multi-region deployment configuration for EU/US data residency	Medium Requires coordinating SSO client credentials per hospital.

ROADMAP READY: Executive Summary: With Phase 1 deployed, the critical security flaw (plaintext exposure) and authentication failure (invalid admin key) are completely neutralized. The roadmap establishes a clear, phased pathway to enterprise multi-tenant maturity.